



HANDLEIDING PRODUCTIE SV-BEVRAGING

Jorre Vannieuwenhuyze 

22 april 2026 

1. ALGEMEEN

- Alle bestanden bevinden zich in de map

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/  
021_SVbevraging
```

Deze map is opgedeeld in drie submappen:

- 01_code: Bevat alle verwerkingscode voor de SV-bevraging. De map wordt gesynchroniseerd met GitHub: <https://github.com/vsa-datascience/SVsurvey.git>

Opmerking 1: naast scripts omvat deze map ook datasets die manueel worden aangemaakt om het productieproces overzichtelijk te houden – een werkwijze die binnen de VSA nog niet standaard is.

Opmerking 2: De codes bevatten nu ook processtappen die op termijn beter op algemeen niveau binnen de VSA door dedicated teams zouden moeten worden beheerd en uitgevoerd. Het gaat hierbij om de aanmaak van algemene programmeerfuncties, het beheer van het datamodel, en de aggregatie van demografische bevolkingsdata voor de berekening van de gewichten.

- 02_sourcedata: Opslagplaats voor alle aangeleverde ruwe datasets, onderverdeeld per versie (SV0001, SV0002, SV0003, ...). Elke versie bevat twee submappen:

- + 01_sample: Steekproefdata van Statbel
- + 02_survey: Enquêtedata van de veldwerkbureaus

Bij meerdere leveringen per versie is het aangeraden de bestanden verder onder te verdelen in mappen per aanleverdatum (formaat: YYYYMMDD).

Deze map bevat ook de submap Demobel-stock, met bevolkingsdata voor het berekenen van populatiestatistieken en weegfactoren voor de enquêterespondenten. Op termijn hoort deze data structureel thuis in //WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/011_Demografie, waarna Demobel-stock overbodig wordt.

- 03_deriveddata: Bevat alle tussentijdse, afgeleide bestanden die uitsluitend intern worden gebruikt tijdens de verwerking.
- Gedeelde data wordt weggeschreven naar twee aparte mappen:

- `//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/998_Scientific_Use_Files`
Data voor intern gebruik binnen de VSA, raadpleegbaar door alle VSA-medewerkers maar niet bestemd voor externe publicatie. Voorbeelden zijn de Scientific Use Files (SUF's).
- `//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_Public_Use_Files`
Data die open gepubliceerd mag worden en ook extern toegankelijk is. Hieronder vallen de Public Use Files (PUF's), zoals de bestanden met de officiële VOS'en.
- Mappen, labelcodes en dergelijke worden standaard in het Engels benoemd. Dit is een goede praktijk die de duurzaamheid en herbruikbaarheid van het werk ten goede komt.
- De meeste databestanden staan momenteel in `.xlsx`-formaat. Gezien de expertise binnen de VSA is dit het meest toegankelijke formaat voor inspectie en manuele aanpassingen, maar het is niet de meest efficiënte keuze. Formaten zoals `.csv`, `.parquet` of `.json` zijn stabiel en performanter. Het is aan het team om deze overstap in de toekomst te overwegen.
- Alle scripts zijn geschreven in R, maar kunnen in principe zonder veel moeite worden omgezet naar Python.
- Het verwerkingsproces start altijd door het RStudio-project `01_code/SVsurvey.Rproj` te openen. Dit stelt de working directory in op de map `01_code`, waarop alle paden in de code relatief zijn.
- Map `01_code/00_functions` bevat een soort R-package met functies voor de dataverwerking. Ze is opgebouwd volgens de algemene principes van een R-package. De functies zijn terug te vinden in sub-map R en werden gedocumenteerd via Roxygen2 in de code zelf.
- Deze documentatie werd aangemaakt in map `01_code/99_documentation`.

2. DATAMODEL

De eerste stap die we zetten is de opzet van een datamodel. We definiëren hiervoor eerst codelijsten. Nadien definiëren we de data structuur definities (DSD's) voor alle Internal, Scientific en Public Use files. In de DSD's wordt een overzicht van kolommen/variabelen gegeven en worden,

indien nodig, codelijsten aan deze kolommen gekoppeld. Hiermee volgen we de principes van SDMX.

Het datamodel wordt opgebouwd in map 01_code/01_data_model. Afgeleide codelijsten en data structuur definities worden weggeschreven naar de map 03_deriveddata/01_codelists. In principe wordt het beheer van het datamodel op termijn best overgedragen naar een dedicated datamodel-team, die werkt in map //WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/002_DataModel.

Het datamodel moet in principe ook enkel worden aangepast als er een nieuwe vraag wordt toegevoegd aan de SV-bevragingen, of als correcties worden gemaakt.

2.1. CODELIJSTEN

De map 01_data_model/01_code_lists bevat drie groepen codelijsten die worden aangemaakt via de codes in de map.

2.1.1. Algemene codelijsten

01_general_codelists: Dit zijn algemene codelijsten die geen speciale behandeling vragen. De aanmaak vraagt echter wel wat uitleg. Heel wat codelijsten zijn immers aan elkaar gelinkt. Eerste voorbeeld: Heel wat vragen in de SV-bevraging hebben identieke niet-substantiële categorieën zoals “Niet van toepassing” of “Ik weet niet/geen antwoord”. We zorgen ervoor dat de labelling van deze categorieën consistent blijft over de codelijsten heen. Tweede voorbeeld: Verschillende codelijsten zijn subsets van andere codelijsten. Voor demografische data hebben we bijvoorbeeld de codelijst “sex” met categorieën “man” en “vrouw”. Dit is een subset van de codelijst “srsex” (self reported sex) met categorieën “man”, “vrouw” en “X” zoals gebruikt in de vragenlijst van de SV-bevraging. Ook hier zorgen we best voor consistentie in labelling.

De general codelists worden daarom aangemaakt vanuit twee bestanden:

- input/code_list_values.xlsx bevat een overzicht van alle waarden die kunnen voorkomen op codelijsten. Dit overzicht bevat
 - een value: een gestandaardiseerde code die wordt gebruikt in datasets en voor scripting om referenties gemakkelijk te maken
 - een description_nl: Een nederlandstalige beschrijving van de waarde. Dit is louter als naslagwerk bedoeld en wordt verder niet gebruikt in de dataverwerking.

- `label_nl`, `label_fr`, `label_en`, ...: Dit zijn Nederlandse, Franse, Engelse, ...labels die worden gebruikt in output zoals public use files of gepubliceerde tabellen. Door deze labels in dit gecentraliseerd overzicht te bewaren zijn aanpassingen gemakkelijk door te voeren.

Eigenlijk is dit een soort conceptenlijst zoals gebruikt in SDMX, maar dan voor waarden op codelijsten. Mijn plan was een algemene conceptenlijst aan te leggen voor zowel variabelen als waarden, omdat bij dataverwerking waarden kunnen worden omgezet naar variabelen wanneer tabellen worden gepivoteerd.

- `input/code_list_skeletons.xlsx`: Dit bestand bevat de structuur van de codelijsten. Elk tabblad geeft een structuur van codelijsten die subsets van elkaar zijn. Het bevat kolommen
 - `value`: sleutel met bestand `code_list_values.xlsx`
 - `valuen`: Een numerieke waarde voor de `value`, zodat categorieën steeds consistent geordend kunnen worden
 - kolommen met aan te maken codelijsten. De kolomnaam is de naam van de codelijst die zal aangemaakt worden. Rijen met waarden `TRUE` bevatten de waarden die opgenomen worden in deze codelijst, rijen met waarde `FALSE` worden genegeerd in deze codelijst.

Script `create_general_code_lists.R` combineert beide bestanden en schrijft de codelijsten weg naar `03_deriveddata/01_codelists`. Hierin komen alle aparte codelijsten te staan als `.xlsx`-bestanden. Elke codelijst bevat de kolommen:

- `valuen` en `value`
- de labels in verschillende talen
- de `values` van codelijsten die een subset zijn. Dit kan gebruikt worden als mapping tijdens de dataverwerking. Zo kan bijvoorbeeld voor de SV-bevraging een variabele met niet-substantiële categorieën (“Weet niet/geen antwoord” etc.) snel gemapt worden naar een codelijst zonder die niet-substantiële categorieën op een consistente manier.

2.1.2. Specifiekere codelijsten

Naast de algemene codelijsten zijn er ook nog heel wat codelijsten met specifiekere verwerking. Deze worden in de andere mappen aangemaakt.

NIS codes

Script `02_NIS_codes/create_code_lists_NIScodes.R` maakt codelijsten aan voor de NIS-codes voor gewesten, provincies, arrondissementen, gemeenten en deelgemeenten. Dit gebeurt op basis van de bestanden van Statbel die kunnen worden gedownload van <https://statbel.fgov.be/nl/over-statbel/methodologie/classificaties/geografie>, terug te vinden in map `02_NIS_codes/input`. Er worden verschillende versies van codelijsten weggeschreven voor de jaren 1983, 1995, 2019 en 2025. In deze jaren was er immers steeds een wijziging in de NIS-codering aangebracht. De codelijsten worden ook steeds aangemaakt voor de verschillende niveau's:

- 0 = het rijk
- 1 = gewesten
- 2 = provincies
- 3 = arrondissementen
- 4 = gemeenten
- 5 = deelgemeenten

In map `03_deriveddata/01_codelists` vind je ze terug als `cl_refnisVERSIONlv1NIVEAU.xlsx`. Daarnaast wordt er voor elke versie een algemene codelijst aangemaakt die de verschillende niveau's combineert `cl_refnisVERSION.xlsx`. In elke codelijst ga je opnieuw kolommen vinden om te mappen naar andere versies en niveau's, bijvoorbeeld om gemeenten om te zetten naar provincies.

Landencodes

Landen worden op verschillende manieren gecodeerd en ingedeeld. Het script `03_country_codes/create_code_lists_countries.R` creëert deze codelijsten. Het gaat hierbij om een indeling volgens ISO2, ISO3, de NIS-codes, en indelingen naar landengroepen.

Andere codelijsten

Voor de SV-bevraging worden niet al deze codelijsten gebruikt. Voor de NIS-codes worden enkel codes gebruikt op gemeenteniveau. Bij de landencodes worden enkel de NIS-landencodes gebruikt. De aanmaak van deze codelijsten werden echter al ontwikkeld binnen het transitietraject voor de VSA-breed. Ik heb dus gewoon deze programma's gekopieerd naar de SV-bevraging.

Daarnaast werkte ik ook al aan andere codelijsten. De programma's hiervoor zijn te vinden in de map `//WV162699/fs_kb_dkb/svr/Beveiligde_`

data_VSA/002_DataModel/00_Code, maar deze vragen nog verdere verwerking. Het gaat hierbij om de creatie van codelijsten voor NIS-codes van de statistische sectoren, voor tijdsvariabelen, voor postcodes en voor andere variabelen van demobel stand van de bevolking. In deze aanmaak zitten geen codelijsten nuttig voor de SV-bevraging maar kunnen wel gebruikt worden later voor andere dataverwerkingsprocessen, wanneer het beheer van de datamodellen gecentraliseerd worden bij één team.

Verdere opermerkingen

- Script `create_code_lists.R` creëert alle codelijsten in `03_deriveddata/01_codelists` van een clean sheet door alle andere scripts te laten lopen in de submappen. Het bevat dus geen code die zelf codelijsten aanmaakt, maar roept alle andere scripts aan. Het voordeel van dit script is dat het eerst de map `03_deriveddata/01_codelists` verwijdt om nadien weer aan te maken.
- In de scripts mankeren nog checks op consistenties binnen en tussen codelijsten. Bijvoorbeeld, zijn alle namen uniek, heeft elke codelijst tenminste een `value` en `label_nl` kolom, zijn de `values` en numerieke `values` uniek, ...Dit kan nog toegevoegd worden.

2.2. DATA STRUCTUUR DEFINITIES

Nadat de codelijsten zijn aangemaakt, kan de data structuur definitie (DSD) worden aangemaakt. In de DSD worden alle kolommen/variabelen van een dataset op gelijst, ook wel concepten genoemd, en eventueel gekoppeld aan codelijsten volgens de principes van SDMX. Op deze manier vormt de DSD de basis voor de sterschema's in onze database. Wij gaan echter een stap verder en voegen ook andere criteria toe voor de concepten, bijvoorbeeld minimum en maximum waarden voor numerieke concepten, of regular expressions voor karakteriële concepten.

De strategie is dat er één grote centrale DSD wordt aangemaakt met een overzicht van alle variabelen in alle mogelijke datasets en datatabellen samen met de bijhorende criteria. Voor specifieke datasets zoals de public use files of de datatabellen met VOS-cijfers worden de gepaste concepten/variabelen geselecteerd en criteria eventueel verfijnd.

De aanmaak van de DSD start van bestand `01_code/01_data_model/02_DSD/dsd_variables.xlsx`. Deze spreadsheet start van een overzicht van alle mogelijke variabelen. Het bevat volgende kolommen:

- `concept_id`: Unieke identificatienaam van de variabele (unieke sleutel / variabelenaam).

- `description_nl`: Uitgebreide inhoudelijke beschrijving van de variabele in het Nederlands, dit wordt verder niet gebruikt in de data-verwerking.
- `label_nl`: Korte, publiekgerichte label van de variabele in het Nederlands.
- `label_fr`: Korte label van de variabele in het Frans.
- `label_en`: Korte label van de variabele in het Engels.
- `format`: Datatype of opslagformaat van de variabele, heeft enkel waarden `character`, `codelist`, `integer`, `numeric` of `date`.
- `codelist`: Verwijzing naar de gebruikte codelijst indien de variabele gecodeerde waarden bevat, de codelijst moet terug te vinden zijn in `03_deriveddata/01_codelists`.
- `regex`: Reguliere expressie waaraan de waarden van de variabele moeten voldoen (validatieregel), enkel voor karakteriële variabelen.
- `min`: Minimale toegelaten waarde (voor numerieke variabelen).
- `max`: Maximale toegelaten waarde (voor numerieke variabelen).
- `missing_values`: Logisch, is `TRUE` als de variabele missende waarden mag bevatten en `FALSE` als er geen missende waarden mogen voorkomen.
- `complete`: Indicator die aangeeft of er in een dataset verwacht wordt dat alle mogelijke waarden voorkomen. Dit wordt gebruikt voor de aanmaak van bijvoorbeeld VOS-tabellen. Als je in zo'n tabel bijvoorbeeld voor elke gemeente een waarde verwacht, moet `complete=TRUE` voor variabele `refnisXXXXlv4`.

Script `01_code/01_data_model/02_DSD/check_and_publish_dsd.R` kopieert `dsd_variables.xlsx` naar `03_deriveddata/02_DSD/dsd.xlsx`, maar voert eerst verschillende checks uit zodat er geen fouten in de DSD kunnen voorkomen. Voorbeelden van zo'n checks zijn nazicht dat alle concept ID's uniek zijn, dat voor elk concept met codelijst een geldige verwijzing gebeurt naar een codelijst in `03_deriveddata/01_codelists`, dat de regular expressions valide zijn, etc. Het bestand `03_deriveddata/02_DSD/dsd.xlsx` wordt dus gebruikt voor de verdere dataverwerking.

2.3. OPMERKINGEN

Mijn strategie was dat in de dataverwerking de DSD verder verfijnd kan worden AANVULLEN

3. POPULATIE INFORMATIE

De tweede stap is de aggregatie van bevolkingsstatistieken voor de berekening van de poststratificatiegewichten in de SV-bevraging. Dit gebeurt op de stock bevolkingsdata. Dit is eigenlijk een proces dat op een gestructureerde manier zou moeten worden opgenomen door de demografen (of de data engineers verantwoordelijk voor de demografiedataprocessen) in folder `//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/011_Demografie`. Waarna de scripts voor de verwerking van de SV-bevraging deze geaggregeerde data daar gewoon kunnen ophalen.

Aggregatie van de bevolkingsdata gebeurt per jaar en niet per editie van de SV-bevraging, aangezien de bevolkingsdata per jaar wordt opgeleverd. Vandaar dat dit een aparte stap is.

De aggregatie gebeurt op basis van de demografische stock data. Deze worden in bewaard in map `021_SVbevraging/02_sourcedata/Demobel-stock`. Steeds in sav-formaat. Gebruik steeds dezelfde naam `"stock_YYYY.sav"` want de scripts halen deze bestanden automatisch op! Als een andere naam wordt gekozen zullen de scripts niet werken. Vraag steeds de demografen om de databestanden aan te leveren, of dus op termijn om zelf de aggregaties gestandaardiseerd te voorzien waardoor deze map met dubbele databewaring kan verwijderd worden.

De aggregatie wordt gedaan via de scripts in map `01_code/02_population_data`. Er is een script per jaar maar deze roepen enkel de functie `SVsurvey_aggregate_population()` uit `01_code/00_functions/R` aan die automatisch de geaggregeerde cijfers wegschrijft naar `03_deriveddata/03_aggregated_population_data`.

4. DATA VERWERKING

De volgende stap is om voor elke editie de data te verwerken. Dit gebeurt in map `01_code/03_process_editions` waarin je een scriptmap vindt per editie en een map voor algemene metadata. De verwerking van de data gebeurt in verschillende stappen.

4.1. METADATA

Op de eerste plaats wordt metadata op een overzichtelijke manier bewaard in databestanden in plaats van programmascripts. Deze metadata

moet doorheen het verwerkingsproces en bij elke editie actief worden aangevuld (anders krijg je wel een foutmelding die aangeeft dat het moet worden aangevuld). Dit zijn de databestanden die worden gebruikt:

- 01_code/03_process_editions/Metadata/Editions.xlsx:
Geeft een overzicht van de verschillende edities van de SV-bevraging.
- 01_code/03_process_editions/Metadata/Mailings.xlsx:
Geeft een overzicht van alle contactpogingen voor elke veldwerk-periode binnen de SV-bevraging.
- 01_code/03_process_editions/Metadata/Questions.xlsx: Dit bevat een overzicht van alle vragen die worden gesteld in de SV-bevraging. Het koppelt de `concept_id`'s uit het datamodel aan specifieke vraagverwoordingen. Die vraagverwoording is terug te vinden in de kolommen `question` en `question_item`. Daarnaast bevat het ook informatie over de vragenlijstmodule waarin de vraag wordt gesteld in kolom `question_module`. Dit bestand kan gebruikt worden om kleine wijzigingen in vraagverwoording te registreren waarbij geen grote resultaatbreuken worden verwacht. Hiervoor maak je gebruik van de kolom `from_surv_id`. In deze kolom staat de versie van de SV-bevraging vanaf wanneer een bepaalde vraagverwoording werd gebruikt. Bijvoorbeeld, in de 11de editie veranderde de vraagverwoording voor de vragen rond sportparticipatie van “Welke sporten beoefent u momenteel?” naar “Welke sporten heeft u de voorbije 12 maanden beoefend?”. Voor elk van deze vragen zal je in de dataset twee rijen vinden, één voor de eerste verwoording vanaf editie SV0001 en één voor de tweede verwoording vanaf editie SV0011.

Opgelet, bij grote wijzigingen in vraagverwoording waarbij een grote impact wordt verwacht op de resultaten, moet er per definitie een nieuw concept worden gedefinieerd in het datamodel. Er dient ook nog nagekeken te worden of alle wijzigingen in vraagverwoordingen van SV0001 tot SV0015 in dit bestand geregistreerd staan.

- 01_code/03_process_editions/Metadata/Answers.xlsx: Dit bevat alle antwoordverwoordingen die worden getoond in de vragenlijst van de SV-bevragingen en in principe worden gebruikt als value label in de aangeleverde data. De value label wordt gekoppeld aan de codelijsten via de unieke value. Net zoals bij Questions.xlsx kan deze dataset gebruikt worden om kleine wijzigingen in verwoordingen te documenteren. Dit gebeurt op dezelfde manier als bij de vragen via de kolom `from_surv_id`. Bij de aanmaak van de codeboeken en in de cleaning wordt steeds gekeken naar de meest re-

cente value label door de hoogst mogelijke editie te nemen onder `from_surv_id` die lager is dan de gevraagde editie.

Opnieuw, bij grote wijzigingen in antwoordverwoording waarbij een grote impact wordt verwacht op de resultaten, moet er per definitie een nieuw concept worden gedefinieerd in het datamodel. Er dient ook nog nagekeken te worden of alle wijzigingen in vraagverwoordingen van SV0001 tot SV0015 in dit bestand geregistreerd staan.

4.2. AANMAAK CODEBOEK

4.3. CLEANING DATA